
ENGENHARIA DE COMPUTAÇÃO

Disciplina: Engenharia de Software

KA 07 — Manutenção de Software

(Software Maintenance)

Plano de Manutenção de Software

Instituição de Ensino	UNOESC
Professor(a) Responsável	Odemir Moreira da Mata Jr.
Disciplina	Engenharia de Software
Turma / Semestre	JBA620-6
Integrantes do Grupo	Ana Clara Viganó Prestes de Oliveira, Davi Antônio Basotto Minati, Elian Baldasso Pereira Duarte, Jasmine Masson Alves, Lucas Eduardo Heydt, Vitória Aparecida Vendausen
Data de Entrega	24/06/2026

Histórico de Versões

Versão	Data	Autor(es)	Descrição	Revisado por
v1.0	29/05/2026	Lucas Heydt	Definição da estratégia de manutenção e suporte.	Elian Duarte
v2.0	24/06/2026	Vitória Vendausen	Consolidação final do documento.	Jasmine Alves

1. Introdução

1.1 Propósito

Este documento define as políticas, processos e responsabilidades para a manutenção do sistema clínico (SisConsult) após sua implantação no ambiente de produção. O objetivo é garantir que o software permaneça seguro, funcional, aderente às legislações de saúde vigentes e sustentável a longo prazo, minimizando a degradação estrutural inerente ao ciclo de vida de aplicações complexas.

1.2 Fundamentação Teórica — Leis de Lehman

A estratégia de manutenção deste projeto fundamenta-se nas Leis da Evolução de Software formuladas por Meir M. Lehman (LEHMAN, 1980). Duas leis são particularmente críticas para o contexto do SisConsult:

- Lei da Mudança Contínua (Primeira Lei):** Um sistema utilizado no mundo real deve mudar e se adaptar, ou se tornará progressivamente menos útil. O ambiente de saúde é dinâmico; portanto, o sistema precisará de manutenções contínuas para não se tornar obsoleto diante das necessidades clínicas.
- Lei da Complexidade Crescente (Segunda Lei):** À medida que o sistema evolui, sua complexidade aumenta, a menos que seja feito um esforço explícito para mantê-la ou reduzi-la. Isso justifica a existência da Seção 4 deste documento (Política de Gestão de Dívida Técnica), que institui práticas de refatoração para evitar que o código se torne um "espaguete" inatingível.

2. Categorias de Manutenção

2.1 Manutenção Corretiva

Ações reativas executadas para corrigir defeitos (bugs) identificados após a entrega do software, visando restaurar o comportamento esperado.

Exemplos no consultório: Correção de um erro HTTP 500 ao tentar salvar o CID em um prontuário; correção de falha na geração do PDF de receitas médicas; ajuste em falha de cálculo na tabela de horários da recepção que gera agendamentos duplicados.

| 2.2 Manutenção Preventiva

Ações proativas realizadas para encontrar e corrigir problemas latentes antes que eles se tornem falhas operacionais, além de garantir a manutenibilidade futura do código.

Exemplos no consultório: Refatoração de um microserviço que está atingindo complexidade ciclomática alta; atualização de bibliotecas de criptografia antes que suas versões atuais sejam descontinuadas; adição de novos testes automatizados a módulos críticos.

| 2.3 Manutenção Adaptativa

Modificações no sistema para acomodar mudanças em seu ambiente operacional externo, sejam elas tecnológicas, mercadológicas ou regulatórias. Exemplos no consultório:

Mudanças Legais (LGPD): Atualização dos algoritmos de anonimização caso a Autoridade Nacional de Proteção de Dados (ANPD) emita novas diretrizes técnicas.

Resoluções do CFM: Adaptação do módulo de prontuário eletrônico para atender a uma nova resolução do Conselho Federal de Medicina (CFM) sobre padrões de assinatura digital e guarda temporal de documentos.

Infraestrutura: Migração do banco de dados para uma nova versão de serviço gerenciado na nuvem (AWS RDS / Azure SQL).

| 2.4 Manutenção Perfectiva

Modificações solicitadas pelos usuários para expandir o escopo original, adicionando novas funcionalidades ou melhorando o desempenho, usabilidade e eficiência geral.

Exemplos no consultório: Criação de um dashboard gerencial de métricas para os sócios da clínica; otimização do tempo de carregamento da lista de pacientes na tela da recepção; criação de atalhos de teclado para médicos preencherem receitas mais rapidamente.

3. Processo de Gestão de Mudanças

| 3.1 Solicitação de Mudança

Toda necessidade de modificação (seja reporte de bug ou nova funcionalidade) deve ser registrada formalmente em um sistema de controle (ex: Jira, ServiceNow) por meio de um *Ticket*. O ticket deve conter a descrição do problema/ideia, passos para reproduzir (em caso de bugs), e o impacto percebido pelo usuário.

| 3.2 Triagem e Priorização

Um comitê composto pelo Tech Lead e pelo Product Owner (PO) avaliará os tickets semanalmente. A triagem classificará a solicitação em uma das quatro categorias de manutenção (Corretiva, Preventiva, Adaptativa, Perfectiva) e avaliará seu custo vs. benefício. Mudanças aprovadas entram para o *Product Backlog*.

| 3.3 Critérios de Urgência para Manutenção Corretiva

Incidentes de produção desviam o fluxo padrão e são tratados imediatamente com base em sua severidade:

- Crítica (P1): Sistema totalmente indisponível ou vazamento de dados sensíveis. Resposta imediata (SLA < 1 hora).
- Alta (P2): Funcionalidade core (ex: salvar prontuário) indisponível sem contorno. SLA < 4 horas.
- Média (P3): Funcionalidade afetada, mas com solução de contorno (*workaround*). Entra na próxima Sprint.
- Baixa (P4): Erros cosméticos, de interface ou de texto. Fica no backlog.

| 3.4 Execução e Validação

A execução segue as políticas estabelecidas no KA 06 (Operações). O código é desenvolvido em uma branch separada (hotfix ou feature), passa por revisão técnica (*Code Review*), testes automatizados no pipeline de CI/CD, e deve ser homologado em um ambiente de *Staging* por um usuário chave (ex: Recepcionista-Chefe ou Médico) antes do *deploy* em produção

4. Política de Gestão de Dívida Técnica

| 4.1 Como Identificar e Registrar Dívida Técnica

A dívida técnica será acompanhada continuamente por meio de mecanismos automatizados e revisões manuais.

No monitoramento automatizado, serão utilizadas ferramentas de análise estática, como o SonarQube, integradas ao pipeline do projeto para identificar problemas como código duplicado, **code smells** e baixa cobertura de testes.

Já no processo manual, durante as revisões de código, os desenvolvedores poderão aprovar implementações que atendam aos requisitos funcionais, mas que ainda necessitem de melhorias futuras. Nesses casos, deverá ser criado um registro específico de dívida técnica, documentando os pontos que deverão ser refatorados em versões posteriores.

| 4.2 Critérios de Priorização

As dívidas técnicas são avaliadas com base na matriz "Juros vs. Esforço". Dívidas que cobram "juros altos" (ex: um componente mal estruturado que precisa ser alterado toda semana, gerando bugs frequentes) possuem prioridade máxima. Dívidas em módulos estáveis que raramente sofrem manutenção têm prioridade baixa.

| 4.3 Proporção Recomendada de Capacidade

Para cumprir a Segunda Lei de Lehman e evitar o colapso estrutural, a equipe adotará a regra de investir 20% da capacidade (pontos de esforço) de cada Sprint exclusivamente no abatimento de dívidas técnicas e manutenções preventivas. Os 80% restantes ficam para manutenções perfectivas (novas features) e corretivas

5. Análise de Impacto de Mudança

| 5.1 Cenário 1: Adição de Módulo de Telemedicina

Componentes afetados: Front-end web (integração com API de WebRTC/Vídeo), API Gateway (novas rotas websocket), Banco de Dados (nova tabela consultas_telemedicina).

Documentos afetados: Arquitetura do Sistema (KA 03), Modelagem de Ameaças (KA 13 - vazamento de vídeo), Casos de Uso (KA 01).

Dados afetados: Tráfego de áudio e vídeo do paciente (Dado Sensível - Art. 5º da LGPD). Requer atualização do Termo de Consentimento.

Estimativa de esforço: Alto. Trata-se de uma manutenção perfectiva de grande porte, assemelhando-se a um novo projeto acoplado ao ecossistema existente.

| 5.2 Cenário 2: Integração com Sistema de Laboratório Externo

Componentes afetados: Criação de um novo microserviço de mensageria (Worker) para consumir APIs externas via cronjob ou webhooks; alteração no módulo de "Exames" para renderizar resultados de terceiros.

Documentos afetados: Diagrama de Integração/Deploy (KA 03), Contratos de API (Swagger).

Dados afetados: PDF de laudos e resultados analíticos (Dados Sensíveis). Necessidade de auditoria sobre como o laboratório parceiro trafega as chaves de integração.

Estimativa de esforço: Médio-Alto. Complexidade focada no tratamento de exceções de rede e adaptação de diferentes formatos de dados (JSON/XML).

Tabelas e Formulários

| Registro de Solicitações de Manutenção (modelo)

ID	Tipo	Descrição	Prioridade	Componente Afetado	Responsável	Status
MAN-001	Corretiva	Erro ao carregar histórico de receitas de meses anteriores (Timeout).	Alta	BD / API Prontuário	Dev Backend	Fechada
MAN-002	Adaptativa	Ajustar formato do CPF no banco conforme nova norma de integração do SUS.	Média	DB / API Pacientes	Dev Fullstack	Em Andamento

MAN-003	Perfectiva	Incluir campo de "Nome Social" na ficha de cadastro da recepção.	Média	Front-end Web	UI/UX / Dev Front	Aberta
MAN-004	Preventiva	Atualizar versão do Node.js de v18 para v22 LTS por fim de suporte.	Baixa	Infra (Dockerfiles)	DevOps	Aberta

Registro de Dívida Técnica (modelo)

ID	Descrição da Dívida	Componente	Impacto	Esforço Estimado	Prioridade	Sprint para Pagamento
DT-001	A rota de busca de horários possui N+1 queries gerando lentidão.	Controller / ORM	Média (Lentidão na recepção)	5 Story Points	Alta	Sprint 14
DT-002	Função de validação de plano de saúde possui complexidade ciclomática 15.	Service (Validações)	Baixo (Código difícil de ler)	3 Story Points	Média	Sprint 16
DT-003	Falta de testes de integração na emissão de atestados médicos.	Test Suite	Alto (Risco de quebra oculta)	8 Story Points	Alta	Sprint 15

Observações e Referências Consultadas

LEHMAN, M. M. (1980): *Programs, life cycles, and laws of software evolution*. Proceedings of the IEEE, 68(9), 1060-1076. Utilizado para fundamentar a inércia da degradação do código e a necessidade vital da manutenção continuada.

ISO/IEC 14764 (Software Engineering — Software Life Cycle Processes — Maintenance): Norma base utilizada para a categorização formal dos quatro tipos de manutenção (Corretiva, Preventiva, Adaptativa e Perfectiva).

SWEBOK v4.0 (Guide to the Software Engineering Body of Knowledge): IEEE Computer Society. Capítulo 5 (*Software Maintenance*), abordando os processos de gestão de requisições, triagem de mudanças e análise de impacto arquitetural.

Lei Geral de Proteção de Dados Pessoais (LGPD): Considerada como diretriz primária e gatilho externo para requisições contínuas de Manutenção Adaptativa.

Resoluções do Conselho Federal de Medicina (CFM): Diretrizes normativas sobre prontuário eletrônico e telemedicina, utilizadas para embasar cenários de impacto de negócio.